



RESEARCH GOAL: REAL-WORLD MALWARE INTELLIGENCE FOR THRAKSHA

I am building Thraksha, an Android security and automation application.

The current prototype already supports:

- Android app inventory
- package identity inspection
- signing-certificate SHA-256 fingerprinting
- base APK SHA-256 hashing
- contextual permission/profile analysis
- signed offline ThreatPack loading
- RSA signature verification of intelligence packs
- network destination metadata analysis
- PolicyEngine decisions
- Advice Mode
- Device Owner containment
- encrypted tamper-evident local audit history

The prototype currently uses controlled demo threat indicators.

I now need a serious research report on how to replace/augment those controlled indicators with real-world, reputable malware/threat intelligence suitable for an Android security product and investor demo.

This is a DEFENSIVE threat-intelligence research task.

Do NOT download malware binaries.

Do NOT provide malware execution instructions, exploit code, payloads, persistence techniques, evasion techniques, or offensive tooling.

Focus on intelligence metadata, IOCs, provenance, licensing, integration, and safe acquisition.

PRIMARY OBJECTIVE

Identify the best real-world threat-intelligence sources from which Thraksha could obtain legitimate:

- SHA-256 malware/APK hashes
- malicious or suspicious signing-certificate fingerprints
- Android malware package identifiers where reliable
- malicious domains
- malicious IP addresses
- IP:port indicators
- malicious URLs
- malware family/classification metadata
- first-seen / last-seen timestamps
- confidence/reputation data
- indicator status / expiry
- provenance/source metadata

Prioritize intelligence useful for Android malware and mobile threats.

SOURCES TO INVESTIGATE

Research at minimum:

- MalwareBazaar / abuse.ch
- ThreatFox / abuse.ch
- URLhaus / abuse.ch
- VirusTotal
- Google/Android malware or Play Protect-related public intelligence where available
- AlienVault OTX if relevant
- MISP-compatible public feeds
- OpenPhish / phishing-oriented feeds if useful
- reputable CERT/government feeds
- reputable Android malware research datasets
- academic Android malware datasets that are still available and legally usable

Also identify better sources I have not listed.

Do not assume a source is suitable simply because it is popular.

FOR EACH SOURCE DETERMINE

Provide:

1. Source/provider name
2. Official website/documentation
3. Intelligence types offered

4. Android/mobile relevance

5. Whether it provides:

- SHA256
- certificates
- package identifiers
- domains
- IPs
- URLs
- malware family
- timestamps
- confidence/reputation

6. Access method:

- downloadable feed
- REST API
- query API
- TAXII
- MISP
- other

7. Authentication requirements

8. Rate limits

9. Freshness/update frequency

10. Historical depth

11. Free/community vs paid/commercial availability

12. Licensing/Terms of Service

13. Whether redistribution is permitted

14. Whether derived/normalized indicators may be redistributed

15. Whether commercial/investor-demo use changes the permission

16. Attribution requirements

17. Data quality/reputation

18. False-positive concerns

19. Staleness concerns

20. Suitability for embedding a small OFFLINE signed ThreatPack

21. Suitability for a future continuously updated production backend

22. Security/privacy risks associated with using the source

Use official documentation whenever possible.

Clearly separate verified facts from inference.

CRITICAL LEGAL / LICENSING RESEARCH

This is especially important.

For each promising source, determine whether Thraksha may legally:

- A. query the source privately;
- B. store returned indicators internally;
- C. normalize indicators into Thraksha's own schema;
- D. include a subset of indicators in an offline APK/demo ThreatPack;
- E. redistribute those indicators to users;
- F. use the source in a commercial product;
- G. display source attribution/classification to users.

If the Terms are ambiguous, explicitly mark:

"LEGAL / LICENSING REVIEW REQUIRED"

Do not assume that publicly accessible data is freely redistributable.

ANDROID-SPECIFIC RESEARCH

Investigate how useful these indicator types really are for Android.

APK SHA-256

Research:

- exact APK hash usefulness
- repackaging limitations
- version churn
- polymorphism/rebuild issues
- base APK vs split APK considerations

Signing certificate fingerprints

Research:

- usefulness for identifying malware campaigns
- certificate reuse
- compromised/stolen certificates
- key rotation

- Android SigningInfo/signing history considerations
- false positives from shared debug/test keys

Package names

Research:

- how reliable they are as malicious indicators
- package-name reuse/spoofing
- usefulness only in combination with signer/hash

Domain / IP indicators

Research:

- fast-flux/cloud/CDN/shared-hosting limitations
- IP reuse
- domain expiry/staleness
- recommended TTL/expiration practices

The report should recommend which indicators Thraksha should trust strongly and which should be treated as weak contextual evidence.

DESIGN A THRAKSHA INTELLIGENCE SCHEMA

Recommend a normalized schema for Thraksha.

Example fields to evaluate:

- "indicatorId"
- "indicatorType"
- "indicatorValue"
- "malwareFamily"
- "classification"
- "platform"
- "severity"
- "confidence"
- "source"
- "sourceReference"
- "firstSeen"
- "lastSeen"
- "expiresAt"
- "revoked"
- "enabled"

- "tags"
- "license"
- "attribution"
- "metadata"

Recommend:

- mandatory fields
- optional fields
- enums
- validation rules
- canonical SHA-256 format
- canonical certificate fingerprint format
- IP/domain normalization
- deduplication strategy

CONFIDENCE AND SOURCE TRUST

Design a practical trust model.

We do NOT want:

"one random feed says bad → automatically quarantine"

Recommend a model such as:

- source trust level
- indicator confidence
- number of corroborating sources
- freshness
- indicator type strength
- contextual findings
- network evidence

Explain how Thraksha might distinguish:

"KNOWN THREAT MATCH"

from:

"HIGH-CONFIDENCE IOC MATCH"

from:

"SUSPICIOUS / REVIEW"

without pretending confidence is a machine-learning probability.

STALENESS / EXPIRATION

Research recommended expiry handling.

Different indicator classes may need different lifetimes.

For example:

- APK SHA256 may remain historically valid
- malicious certificate may remain relevant longer
- IP indicators can become stale quickly
- domains may change ownership
- URLs can disappear

Recommend expiration/aging policy by indicator type.

INVESTOR DEMO DATASET

Design a SAFE real-intelligence dataset for an investor demo.

Requirements:

- NO malware binaries
- metadata/IOCs only
- reputable sources
- traceable provenance
- licensing permits intended use
- small enough for offline APK use
- real-world indicators, not fabricated values

Recommend an approximate size such as:

- number of APK hashes
- certificate fingerprints
- domains
- IPs
- package indicators

The investor demo should be able to say truthfully:

"Thraksha is scanning against real-world threat intelligence."

Explain exactly what would make that statement defensible.

IMPORTANT DEMO CONSTRAINT

Do NOT design the demo around guaranteeing that a random clean phone contains malware.

Instead, recommend how to demonstrate:

1. scan real installed apps;
2. compare real hashes/certificates against genuine intelligence;
3. report zero known matches if none exist;
4. separately demonstrate controlled VillainCaller detection;
5. show that the same scanner is using a real intelligence corpus.

Explain why this is more credible than forcing a malware detection.

FUTURE PRODUCTION PIPELINE

Design a future architecture:

"External Intelligence Sources"

- ingestion
- provenance validation
- schema normalization
- deduplication
- confidence calculation
- expiration/revocation
- Thraksha intelligence database
- compact mobile ThreatPack generation
- cryptographic signing
- secure distribution
- device signature verification
- atomic activation
- rollback to previous known-good pack
- audit update.

Research whether STIX/TAXII or MISP should be used on the backend and whether the mobile device should receive a simpler compact format.

MOBILE PERFORMANCE

Recommend how much intelligence should reasonably live on-device.

Compare:

- JSON
- SQLite
- Bloom filter
- hash set/index
- binary format
- segmented packs

Consider:

- APK size
- RAM
- lookup speed
- startup time
- incremental updates

Do not prematurely optimize, but provide a plausible production path.

SECURITY OF THE INTELLIGENCE SUPPLY CHAIN

Research and recommend defenses against:

- compromised threat feed
- poisoned indicators
- MITM
- corrupted pack
- rollback to old intelligence
- forged pack
- malicious/stale source
- accidental false-positive mass quarantine

Thraksha already supports signed packs.

Recommend:

- signing key hierarchy
- offline root key vs online signing key
- version monotonicity
- expiry
- rollback prevention
- emergency revocation
- staged activation
- source provenance
- kill switch for bad intelligence

REQUIRED FINAL OUTPUT

Produce a report structured as:

1. Executive Recommendation
2. What Thraksha Needs From Threat Intelligence

3. Source Comparison Table
4. Detailed Review of Each Source
5. Best Sources for Android Malware
6. Best Sources for Network IOCs
7. Licensing / Redistribution Analysis
8. Android Indicator Reliability Analysis
9. Recommended Thraksha IOC Schema
10. Confidence / Source Trust Model
11. Staleness / Expiration Policy
12. Safe Investor Demo Dataset
13. Production Intelligence Pipeline
14. Secure Update / Signing Architecture
15. Performance / On-Device Storage Strategy
16. Risks and Failure Modes
17. Recommended Phase-10 Assets
18. Exact Download/API Acquisition Plan
19. What NOT to Download
20. Final Ranked Recommendation

FINAL RANKING

At the end provide three concrete stacks:

STACK A — FREE / INVESTOR DEMO

Exact sources and assets to use immediately.

STACK B — EARLY PRODUCTION

What to use when Thraksha begins real user testing.

STACK C — COMMERCIAL SCALE

What would likely require paid/commercial threat-intelligence partnerships.

For each stack specify:

- source
- indicator types
- acquisition method
- licensing caveat
- how often to update

- whether data may be redistributed
- whether it belongs on-device or backend-only.

IMPORTANT RULES

Use current web research.

Prefer primary/official documentation.

Cite every important claim.

Do not invent API capabilities or license rights.

Do not fabricate malware indicators.

Do not download malware.

Do not execute malware.

Do not recommend using live malware binaries for this demo.

Where licensing or redistribution rights are unclear, flag them rather than assuming permission.

The goal is to determine:

What real-world threat intelligence can Thraksha safely, legally, credibly and technically use to turn the existing scanner architecture into a genuine real-world security capability?

Here's the structured research report you asked for on real-world threat intelligence for Thraksha, including source analysis, Android-specific indicator reliability, schema and trust model design, pipeline architecture, and demo strategy. It covers all 20 sections and the three final stacks in detail.

*
**

1. <https://bazaar.abuse.ch/api/>
2. <https://www.kaggle.com/shashwatwork/android-malware-dataset-for-machine-learning/tasks>
3. <https://arxiv.org/abs/2307.16852v1>
4. <https://docs.virustotal.com/docs/api-overview>
5. <https://docs.virustotal.com/reference/public-vs-premium-api>
6. <https://openphish.com/terms.html>
7. https://openphish.com/academic_use.html
8. <https://bazaar.abuse.ch/export/csv/cscb/>
9. <https://pubmed.ncbi.nlm.nih.gov/38020437/>
10. <https://abuse.ch/blog/introducing-threatfox/>
11. <https://tweetfeed.live/threat-intelligence-guide/>

12. <https://eugit.opencloud.lu/MISP/MISP/commit/b1f4b4cf25c955d0974c076b8ccf2e42149b92b7?style=split&whitespace=ignore-eol&show-outdated=>
13. <https://threatfox.abuse.ch/faq/>
14. <https://hodigital.blog.gov.uk/wp-content/uploads/sites/161/2020/03/Cyber-Threat-Intelligence-A-Guide-For-Decision-Makers-and-Analysts-v2.0.pdf>
15. <https://github.com/abusech/ThreatFox>
16. <https://urlhaus.abuse.ch/downloads/text/>
17. <https://publicapi.dev/urlhaus-api>
18. https://www.servicenow.com/docs/r/security-management/threat-intelligence-security-center/view-oob-misp-feeds.html?contentId=dm_lF0fKBbMaZGVu~nfQKw
19. <https://docs.virustotal.com/reference/overview>
20. https://otx.alienvault.com/assets/static/external_api.html
21. <https://github.com/AlienVault-OTX/OTX-Node-SDK>
22. <https://dlthub.com/context/source/alienvault-open-threat-exchange-otx>
23. <https://openphish.com/kb.html>
24. <https://malwareintel.es/blog/cti-inteligencia/feeds-tecnicos-abuse-otx-misp-kev/>
25. https://openphish.com/phishing_feeds.html
26. https://openphish.com/phishing_database.html
27. <https://www.misp-project.org/misp-training/a.3-misp-feed.pdf>
28. <https://www.cert.ssi.gouv.fr/ioc/feed/>
29. <https://info.publicintelligence.net/NCCIC-MalwareIPs-1.pdf>
30. <https://threatfilter.dev/sources/>
31. <https://drebin.mlsec.org/>
32. <https://www.kaggle.com/datasets/l7032681/static-and-dynamic-features-adv>
33. <https://androzoo.uni.lu/>
34. <https://pmc.ncbi.nlm.nih.gov/articles/PMC8483345/>
35. <https://bazaar.abuse.ch/faq/>
36. https://orbilu.uni.lu/bitstream/10993/49254/1/TOPS_Deep_Dive_DREBIN_final.pdf
37. <https://github.com/abusech/MalwareBazaar/>
38. <https://www.sumologic.com/help/docs/platform-services/automation-service/app-central/integrations/malware-bazaar/>
39. <https://publicapi.dev/malware-bazaar-api>
40. <https://abusech-docs.spamhaus.com/>
41. https://github.com/abusech/ThreatFox/blob/main/threatfox_submit-ioc.py
42. <https://kravensecurity.com/threat-intelligence-with-misp-part-4-using-feeds/>
43. <https://threatfox.abuse.ch/>
44. <https://joey727-phisherman.mintlify.app/concepts/threat-feeds>
45. <https://docs.virustotal.com/>

46. <https://bazaar.abuse.ch/export/>

47. <https://docs.virustotal.com/docs/please-give-me-an-api-key>

48. <https://github.com/certtools/intelmq-feeds-documentation/blob/master/OpenPhish/Phishing.md>

49. <https://docs.eclecticiq.com/extensions/current/integrations/openphish/incoming-openphish/>

50. <https://www.mintlify.com/asluppiter/Somnium/tests/phishing-urls>

51. <https://cert.europa.eu/static/threat-intelligence/TLP-WHITE-CERT-EU-MEMO-190607-2.pdf>

52. <https://pmc.ncbi.nlm.nih.gov/articles/PMC9663591/>

53. <https://www.usenix.org/system/files/conference/cset17/cset17-paper-gagnon.pdf>

54. https://developers.google.com/android/binary_transparency/google_apk/verification_details

55. <https://ismalicious.com/posts/building-ioc-pipelines-operational-threat-intelligence>

56. <https://pmc.ncbi.nlm.nih.gov/articles/PMC9044361/>

57. <https://www.cert.ssi.gouv.fr/ioc/CERTFR-2022-IOC-001/>

58. <https://lilicoding.github.io/papers/li2019rebooting.pdf>

59. <https://www.netresec.com/?page=Blog&month=2025-11&post=Optimizing-IOC-Retention-Time>

60. <https://nhimg.org/faq/what-breaks-when-ioc-operationalization-is-weak/>

61. <https://cybersecuritynews.com/threat-intelligence-ioc-lifecycle/>

62. <https://ismalicious.com/posts/operational-threat-intelligence-ioc-prioritization-guide>

63. <https://docs.opencti.io/5.11.X/usage/indicators-lifecycle/>

64. <https://gtidocs.virustotal.com/docs/google-threat-intelligence-indicator-score>

65. <https://bazaar.abuse.ch/>

66. <https://apis.io/providers/urlhaus/>

67. <https://thehive-project.github.io/Cortex-Analyzers/analyzers/MalwareBazaar/>

68. <https://threatfox.abuse.ch/export/>

69. <https://apis.io/apis/malwarebazaar/malwarebazaar/>

70. https://helpcenter.threatq.com/assets/PDFs/Actions/abusech_ThreatFox_Action_Guide_v1.0.2.pdf

71. <https://www.misp-project.org/feeds/>

72. <https://github.com/doomedraven/VirusTotalApi>

73. <https://www.gomisp.com/post/how-to-import-public-threat-intelligence-feeds-into-your-misp-instance>

74. <https://docs.virustotal.com/docs/api-scripts-and-client-libraries>

75. <https://github.com/ririhedou/virustotal-api>

76. <https://github.com/AlienVault-OTX/OTX-Python-SDK/blob/master/OTXv2.py>

77. <https://developers.google.com/android/play-protect/warning-strings>

78. <https://source.android.com/docs/security/bulletin/2026/2026-03-01>

79. <https://github.com/Malware-Hunter/MH-100K-dataset>

80. <https://support.google.com/googleplay/answer/2812853?hl=en>

81. <https://developers.google.com/android/play-protect>

82. <https://sites.google.com/view/casandrantu/dataset>

83. <https://source.android.com/docs/security/bulletin>
84. <https://pubmed.ncbi.nlm.nih.gov/36376412/>
85. <https://nzjohng.github.io/publications/papers/msr2020.pdf>
86. <https://pdfs.semanticscholar.org/5ae4/3536e07e4b5e364fc2a2b4bdaca711ccc6a5.pdf>
87. https://people.ece.ubc.ca/mjulia/publications/On_Benign_Features_in_Malware_Detection_2020.pdf
88. <https://github.com/yueyueL/XAIforAndroidMalware>
89. <https://developers.google.com/android/play-protect/warning-dev-guidance>
90. <https://huggingface.co/datasets/hendriow/mh100k/blob/main/README.md>
91. <https://www.zdnet.com/article/google-play-protect-analyzes-every-android-app-that-it-can-find-on-the-internet/>
92. <https://openphish.com/>
93. <https://openphish.com/index.html>
94. <https://phishunt.io/api/>
95. <https://threatfox.abuse.ch/browse/tag/feed/>
96. <https://github.com/marjatech/threatfox2misp>
97. <https://ieee-dataport.org/documents/dataset-android-malware-detection>
98. https://www.impactcybertrust.org/dataset_view?idDataset=1372
99. <https://github.com/vannu07/Android-Malware-Detection>
100. https://kqlsearch.com/query/Malwarebazaar_Certificate_Blocklist_Detection&cm3bnxn4r0018mc0tixrlb7da